Authentication Flow & Troubleshooting Log

This document tracks recent changes, problems faced, and the solutions implemented across the SafeRide Auth ecosystem.

1. The Missing JWT & Broken OIDC Callback

Problem: After successfully submitting an OTP during login, the browser would transition to the `/home` screen, but the `access_token` and `refresh_token` cookies were completely missing. The backend `/callback` was never being hit.

Root Cause: The backend `VerifyOTP` function was successfully returning a JSON response containing an OAuth `redirectTo` string (pointing to Hydra's consent flow). However, the React `AuthContext.tsx` was intercepting the "success" flag, ignoring the `redirectTo` instruction, and manually forcing the React Router to navigate to `/home`. Because the browser never followed the backend's redirect, the final OAuth `/callback` (which provisions the cookies) never executed.

Solution: Updated `verifyOtp` in `AuthContext.tsx` to detect if `res.data.redirectTo` is present. If it is, the frontend executes `window.location.href = res.data.redirectTo`, halting the React SPA and allowing the browser to properly follow the OIDC HTTP redirects.

---

2. Flash Cookie "login_challenge" Exposure

Problem: During the Hydra OIDC login flow, a `login_challenge` string is generated. If we append this to the frontend URL as a query parameter (e.g. `?login_challenge=xyz`), the URL becomes messy and confusing for the user.

Solution (The Flash Cookie Pattern): The backend sets the `login_challenge` inside a temporary, non-HttpOnly browser cookie and redirects the user to a completely clean URL: `/login`. When the React `<LoginPage>` mounts, it reads the cookie into state memory, and then instantly executes: `document.cookie = "login_challenge=; path=/; expires=Thu, 01 Jan 1970 00:00:00 UTC;"` This instantly destroys the cookie to prevent stale reuse while preserving a pristine URL in the address bar.

---

3. Global 401 Interceptor (Infinite Loop on Login Page)

Problem: We wanted to redirect users to the domain login URL (`/api/v1/auth/oauth/login`) whenever a session expired (resulting in a `401 Unauthorized` API response). However, placing this interceptor on `/auth/me` (which

checks initial session state) caused an infinite redirect loop when an unauthenticated user naturally landed on the `/login` page.

Solution: Isolated the 401 interceptor strictly to **application data endpoints** (e.g., `/telemetry/live`, `/reports`, `/settings`). All `/auth/*` endpoints (like `/auth/me` or `/auth/login`) are excluded from the interceptor. If `/auth/me` returns a 401, the React app safely catches it and keeps the user on the `/login` screen without triggering a backend OAuth redirect loop.

---

4. Multi-Tab Logout Sync (Reverted)

Problem: We wanted to ensure that if a user has 5 tabs open and clicks "Logout" on one, the other 4 log out automatically. The initial implementation used a `storage` event listener bound to the `saferide_session` key.

Status: This implementation was **reverted** upon request. The application instead relies on the global 401 interceptor (see #3). If Tab A logs out (destroying the secure cookie), any subsequent data fetch by Tab B will fail with a 401, instantly kicking Tab B out to the login screen securely via the backend.

---

5. Security Threat Model & Mitigations

Below are the specific cyberattacks the SafeRide authentication system is architected to defeat:

A. Cross-Site Scripting (XSS)

- **How it happens:** An attacker injects malicious JavaScript into the React frontend (e.g., via a compromised npm package or text input) to scrape and steal session tokens.
- **How we tackle it:** We completely avoid storing JWTs in `localStorage`. Instead, we store the `access_token`, `refresh_token`, and `session_id` in **HttpOnly cookies**. The browser guarantees that no JavaScript (not even yours) can read an HttpOnly cookie, making token theft via XSS impossible.

B. Session Fixation

- **How it happens:** An attacker forces a victim's browser to use a known `session_id` (e.g., by sending them a malicious link). The victim logs in, elevating that session's privileges, and the attacker immediately hijacks the account using the session ID they provided.
- **How we tackle it:** We strictly use server-generated cookies rather than URL parameters for session binding. Furthermore, upon a successful OTP verification, the pre-auth session is abandoned and a completely fresh `session_id` is cryptographically generated for the authenticated state.

C. Timing Attacks (Side-Channel)

- **How it happens:** An attacker guesses an OTP (e.g., `123456`). If the first digit `1` is correct, the server's string comparison function takes 1 millisecond longer to fail than if it were completely wrong. The attacker measures this microscopic delay to guess the entire code digit-by-digit.

- **How we tackle it:** The backend uses **Constant-Time Comparison** (`subtle.ConstantTimeCompare` in Go) to verify OTP hashes. The CPU takes the exact same number of clock cycles to verify the OTP regardless of how many digits are correct, completely blinding the attacker.

D. Brute Force & Credential Stuffing

- **How it happens:** A botnet spams the `/api/v1/auth/login/verify` endpoint thousands of times a second to brute-force a user's 6-digit OTP before it expires.
- **How we tackle it:** We implemented a strict **IP/Session-based Rate Limiter** limiting all `/login` and `/register` endpoints to a maximum of 30 requests per minute per user. Any brute-force attempt is instantly blocked with a `429 Too Many Requests` error.

E. Cross-Site Request Forgery (CSRF)

- **How it happens:** An attacker tricks a victim into clicking a link on `evil.com`, which automatically sends a hidden POST request to `localhost:5173/api/v1/auth/delete-account`. Because the user is logged in, their browser blindly attaches their session cookies to the attack request.
- **How we tackle it:** We configure our cookies with the `SameSite=Lax` (or `Strict`) attribute. This instructs the browser to never attach SafeRide cookies to cross-origin background POST requests, neutralizing the attack. Additionally, the OIDC flow uses a cryptographic `state` and `nonce` parameter to verify the origin of all OAuth callbacks.

F. Man-in-the-Middle (MitM) Attacks & Code Interception

- **How it happens:** An attacker intercepts the network traffic between the browser and the server to steal the OAuth authorization code.
- **How we tackle it:** We enforce the `Secure` flag on cookies (requiring HTTPS), and our OAuth flow strictly utilizes **PKCE** (Proof Key for Code Exchange). Even if an attacker steals the authorization code in transit, they cannot exchange it for a token without the cryptographically generated `code_verifier` that only your backend possesses.

G. Open Redirect Attacks

- **How it happens:** An attacker crafts a malicious URL like `https://yoursite.com/login?redirectTo=http://evil-phishing-site.com`. When the user logs in, they are instantly redirected to the fake site and tricked into giving up more information.
- **How we tackle it:** The backend strictly controls all OAuth redirect URLs using internal logic and pre-approved whitelists in the Ory Hydra database. The frontend never blindly follows a user-provided URL parameter.

H. SQL Injection (SQLi)

- **How it happens:** An attacker types raw SQL commands (like `' OR 1=1; DROP TABLE users;`) into the email login field to manipulate the database.
- **How we tackle it:** Our Golang backend utilizes modern parameterized queries/ORMs. User input is treated strictly as data strings and never evaluated as executable SQL code.

I. Replay Attacks

- **How it happens:** An attacker intercepts a valid login request or OTP verification packet and resends (replays) it 10 minutes later to gain unauthorized access.
- **How we tackle it:** OTPs are single-use and strictly bound to short-lived time windows (e.g., 5 minutes) via Redis. Furthermore, the OIDC flow generates a cryptographic `nonce` (Number Used Once) that must exactly match the signature inside the final ID Token, ensuring the login sequence cannot be replayed.